

Ранжирование

Gigaschool course

Александр Потехин

- NLP Team Lead X5
- HSE & ITMO Tutor
- PyConf, AiConf, DataFest speaker



Почему простого поиска недостаточно?

Содержание:

- Пример неудачного поиска
- Запрос: "как обучить нейросеть"
- Retrieval вернул: 100 документов
- Проблема: Топ-5 содержит нерелевантные результаты
 - Позиция 1: "История нейронных сетей" (общая статья)
 - Позиция 2: "Нейробиология мозга" (не про ML!)
 - Позиция 3: "PyTorch tutorial" (релевантно!)
 - Позиция 4: "Нейросети в медицине" (слишком специфично)
 - Позиция 5: "Книга про Deep Learning" (релевантно!)
- Вопрос: Как выбрать лучшие из найденных и поставить их выше?

Двухэтапная архитектура: Retrieval → Reranking

Этап 1: Retrieval (First-stage)

- Вход: Query от пользователя
- Процесс: Быстрый поиск в большой коллекции (миллионы документов)
- Методы: BM25, векторный поиск (ANN)
- Выход: 50-100 кандидатов
- Скорость: ~10-50ms
- Цель: Высокий recall

Этап 2: Reranking (Second-stage)

- Вход: Query + топ-N кандидатов (50-100)
- Процесс: Точная оценка релевантности каждого документа
- Методы: Learning to Rank, Cross-encoders
- Выход: 5-10 финальных документов
- Скорость: ~50-200ms
- Цель: Высокий precision

Метрики качества ранжирования

Основные категории метрик:

- Set-based: Precision@k, Recall@k, F1@k
 - Оценивают только наличие релевантных в топ-k
 - Не учитывают порядок
- Rank-based: MRR, MAP
 - Учитывают позицию релевантных документов
 - MRR - позиция первого релевантного
 - MAP - средняя точность по всем позициям
- Graded relevance: NDCG, ERR
 - Учитывают градации релевантности (0, 1, 2, 3)
 - NDCG - стандарт в современном IR
- Coverage: Hit Rate@k
 - Хотя бы один релевантный в топ-k

Precision@k и Recall@k

$$\text{Recall@k} = \frac{|\{\text{relevant documents in top } k\}|}{|\{\text{total relevant documents}\}|}$$

$$\text{Precision@k} = \frac{|\{\text{relevant documents in top } k\}|}{k}$$

Запрос: "машинное обучение PyTorch"

- **Всего релевантных в коллекции:** 10 документов
- **Топ-5 результатов:** [✓, ✗, ✓, ✓, ✗]
 - Позиция 1: PyTorch tutorial ✓
 - Позиция 2: NumPy guide ✗
 - Позиция 3: PyTorch examples ✓
 - Позиция 4: Deep Learning book ✓
 - Позиция 5: Pandas docs ✗
- **Precision@5 = 3/5 = 0.6** (60% из топ-5 релевантны)
- **Recall@5 = 3/10 = 0.3** (нашли 30% от всех релевантных)

Mean Reciprocal Rank (MRR)

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$$

Mean Reciprocal Rank (MRR)

Reciprocal Rank (RR) для одного запроса:

- Первый релевантный на позиции 1 $\rightarrow RR = 1/1 = 1.0$
- Первый релевантный на позиции 2 $\rightarrow RR = 1/2 = 0.5$
- Первый релевантный на позиции 3 $\rightarrow RR = 1/3 = 0.33$
- Первый релевантный на позиции 10 $\rightarrow RR = 1/10 = 0.1$
- Нет релевантных в топ-k $\rightarrow RR = 0$

Пример расчета для 3 запросов:

- **Query 1:** "что такое BERT"
 - Результаты: [X, X, ✓, X, ✓]
 - Первый релевантный на позиции 3 $\rightarrow RR_1 = 1/3 = 0.33$
- **Query 2:** "как обучить GPT"
 - Результаты: [✓, ✓, X, ✓, X]
 - Первый релевантный на позиции 1 $\rightarrow RR_2 = 1/1 = 1.0$
- **Query 3:** "transformer архитектура"
 - Результаты: [X, ✓, ✓, ✓, X]
 - Первый релевантный на позиции 2 $\rightarrow RR_3 = 1/2 = 0.5$

$$MRR = (0.33 + 1.0 + 0.5) / 3 = 0.61$$

Mean Average Precision (MAP)

$$\text{MAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i$$

$$\text{AP}_i = \frac{1}{R_i} \sum_{k=1}^n P_i(k) \times \text{rel}_i(k)$$

Mean Average Precision (MAP)

Пример расчета для одного запроса:

Запрос: "python data science"

- Всего релевантных: 4 документа
- Результаты:
 - Позиция 1: Pandas guide ✓ → $P(1) = 1/1 = 1.0$
 - Позиция 2: Java tutorial ✗ → пропускаем
 - Позиция 3: NumPy docs ✓ → $P(3) = 2/3 = 0.67$
 - Позиция 4: C++ guide ✗ → пропускаем
 - Позиция 5: Scikit-learn ✓ → $P(5) = 3/5 = 0.6$
 - Позиция 6: Matplotlib ✓ → $P(6) = 4/6 = 0.67$

$$[AP = \frac{1.0 + 0.67 + 0.6 + 0.67}{4} = 0.735]$$

Пример для трех запросов:

- Query 1: $AP = 0.735$
- Query 2: $AP = 0.833$
- Query 3: $AP = 0.611$
- $MAP = (0.735 + 0.833 + 0.611) / 3 = 0.726$

Normalized Discounted Cumulative Gain (NDCG)

$$DCG@K = \sum_{k=1}^K \frac{2^{r^{true}(\pi^{-1}(k))} - 1}{\log_2(k+1)}.$$

$$IDCG@K = \sum_{k=1}^K \frac{1}{\log_2(k+1)}$$

$$nDCG@K = \frac{DCG@K}{IDCG@K},$$

Normalized Discounted Cumulative Gain (NDCG)

Зачем нужна NDCG?

- Precision/Recall/MAP: релевантность = 0 или 1 (бинарная)
- В реальности: документы могут быть "очень релевантны", "частично релевантны", "слабо релевантны"
- NDCG учитывает градации релевантности

Почему нормализация?

- DCG зависит от количества релевантных документов
- NDCG всегда в диапазоне $[0, 1]$
- $NDCG = 1.0$ только при идеальном ранжировании

Hit Rate

Hit Rate =

$$\frac{\text{Number of Queries with at least one relevant document retrieved}}{\text{Total Number of Queries}}$$

Сравнение метрик

Метрика	Use Case	Преимущества	Недостатки
Precision@k	Оценка топ-k	Простота, интуитивность	Не учитывает позицию
Recall@k	Полнота retrieval	Понятная интерпретация	Зависит от общего числа
MRR	QA, entity search	Фокус на первом ответе	Игнорирует остальные
MAP	Библиотеки, архивы	Учет всех релевантных	Бинарная релевантность
NDCG@k	Web search, RAG	Градации + позиция	Сложность расчета
Hit Rate@k	A/B тесты, мониторинг	Быстро вычислить	Не учитывает позицию

Традиционные методы ранжирования

Содержание:

Что такое традиционные методы?

- Методы основанные на статистике терминов
- Не требуют обучения (unsupervised)
- Работают на уровне токенов/слов
- Основаны на лексическом совпадении

Основные представители:

1. **TF-IDF** (Term Frequency - Inverse Document Frequency)
 - Классика IR с 1970-х
 - Базируется на частоте термина
2. **BM25** (Best Matching 25)
 - Улучшенная версия TF-IDF
 - Учитывает длину документа
 - Стандарт в поисковых системах

Learning to Rank: переход от heuristics к ML

Почему нужен Learning to Rank?

Проблемы традиционных методов:

- BM25/TF-IDF: фиксированная формула
- Не учитывают дополнительные features (PageRank, freshness, user signals)
- Нет адаптации к конкретному домену
- Один размер не подходит всем

Что такое Learning to Rank (LTR)?

- Применение ML для обучения функции ранжирования
- Вход: Query-document пары + features
- Выход: Relevance score или ranking
- Обучение: На labeled data (qrels - relevance judgments)

Learning to Rank: переход от heuristics к ML

Три парадигмы LTR:

1. Pointwise (Regression/Classification)

- Предсказываем абсолютный score для каждого документа
- Loss: MSE, Cross-Entropy
- Пример: Linear Regression, Neural Network

2. Pairwise (Preference Learning)

- Учимся предпочитать один документ другому
- Loss: Pairwise hinge loss, RankNet loss
- Пример: RankNet, RankSVM, LambdaRank

3. Listwise (Direct Optimization)

- Оптимизируем ranking метрику напрямую
- Loss: Approximation of NDCG/MAP
- Пример: ListNet, LambdaMART, AdaRank

Pointwise Learning to Rank

Основная идея:

- Рассматриваем каждый query-document pair независимо
- Предсказываем relevance score или класс
- Regression: score $\in \mathbb{R}$
- Classification: rel $\in \{0, 1, 2, 3\}$

Предсказываем relevance score или класс

Regression: score $\in \mathbb{R}$

Classification: rel $\in \{0, 1, 2, 3\}$

Рассматриваем каждый query-document pair независимо

Пример pipeline:

1. Feature extraction: [BM25=0.8, PageRank=0.3, Length=500, ...]
2. Model prediction: score = 2.3
3. Ranking: sort by score

Pointwise Learning to Rank

Преимущества:

- Простота реализации
- Много готовых ML алгоритмов
- Можно использовать любую regression/classification модель

Недостатки:

- Не учитывает относительный порядок
- Одинаковый score для query1 и query2 означает разное
- Не оптимизирует ranking metrics напрямую
- Менее эффективен чем pairwise/listwise

Pairwise Learning to Rank

Основная идея:

- Вместо абсолютного score учимся сравнивать пары
- "Документ А релевантнее документа В"
- Минимизируем число неправильных пар

Pairwise Learning to Rank

Преимущества:

- Фокус на относительном порядке
- Более естественно для ранжирования
- Лучше чем pointwise на практике
- Много успешных реализаций

Недостатки:

- $O(n^2)$ пар для обработки
- Все пары одинаково важны (но см. LambdaRank)
- Не оптимизирует NDCG напрямую

Listwise Learning to Rank

Основная идея:

- Оптимизируем ranking метрику напрямую
- Рассматриваем весь список документов целиком
- Наиболее близко к реальной задаче ранжирования

Зачем нужен listwise?

- Pointwise/Pairwise не оптимизируют NDCG/MAP напрямую
- Ranking метрики определены на уровне списка
- Можем учитывать position bias

Deep Learning ranking

Эволюция подходов:

- 2000s: Traditional methods (BM25)
- 2005-2010: Learning to Rank (LambdaMART)
- 2013+: Neural methods (Word2Vec, BERT)
- 2018+: Transformer-based ranking

Почему нейронные методы?

Проблемы LTR:

- Feature engineering is manual and expensive
- Не используют семантическую информацию
- Vocabulary mismatch problem

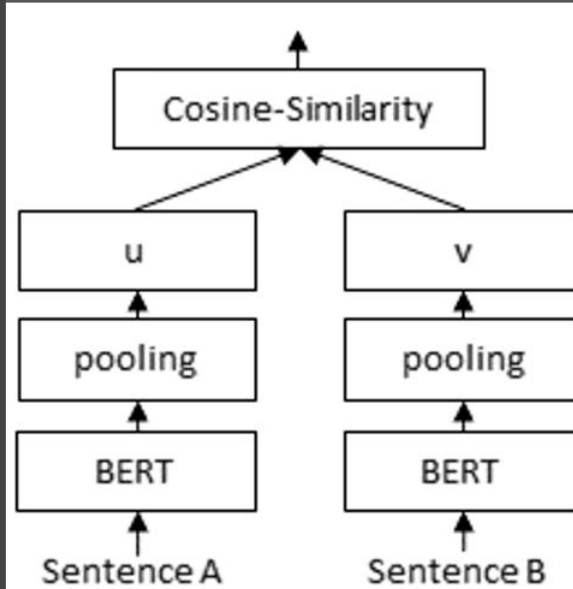
Deep Learning ranking

Что дают нейронные методы:

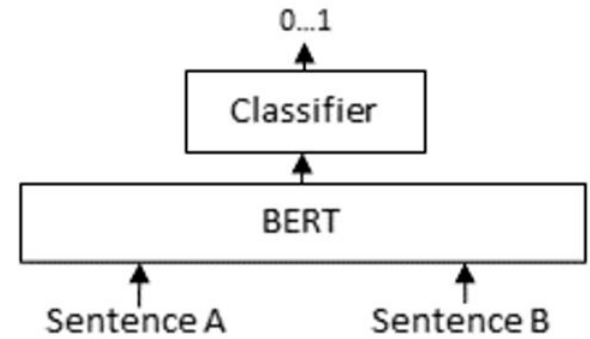
- Автоматическое изучение representations
- Semantic understanding (синонимы, парафразы)
- Transfer learning (pre-training)
- End-to-end обучение
- Multimodal возможности

Encoders

Bi-Encoder



Cross-Encoder



Encoders

1. Bi-encoder (Two-tower)

- Независимое кодирование
- Можно pre-compute embeddings
- Быстрый ANN search
- Используется для retrieval (первый этап)

1. Cross-encoder (Interaction-based)

- Joint encoding с full attention
- Точнее, но медленнее
- Используется для reranking (второй этап)

Bi-encoder

Ключевые особенности:

- Independent encoding: query и doc кодируются отдельно
- Pre-computation: можно заранее закодировать все документы
- ANN search: используем FAISS, Annoy для быстрого поиска
- Scalability: работает на миллионах документов

Training:

Contrastive learning:

- Positive pair: (query, relevant_doc)
- Negative pairs: (query, irrelevant_docs)
- Loss: InfoNCE, Triplet loss, Multiple Negatives Rankin

Cross-encoders

Ключевое отличие от Bi-encoder:

- **Joint encoding:** query и doc обрабатываются вместе
- **Full attention:** каждое слово query видит каждое слово doc
- **No pre-computation:** нужно кодировать каждую пару заново

Training:

- **Pointwise:** Классификация "релевантен/не релевантен" для каждой пары
- **Pairwise:** Сравнение пар документов, какой более релевантен
- **Данные:** MS MARCO (запросы + релевантные документы)

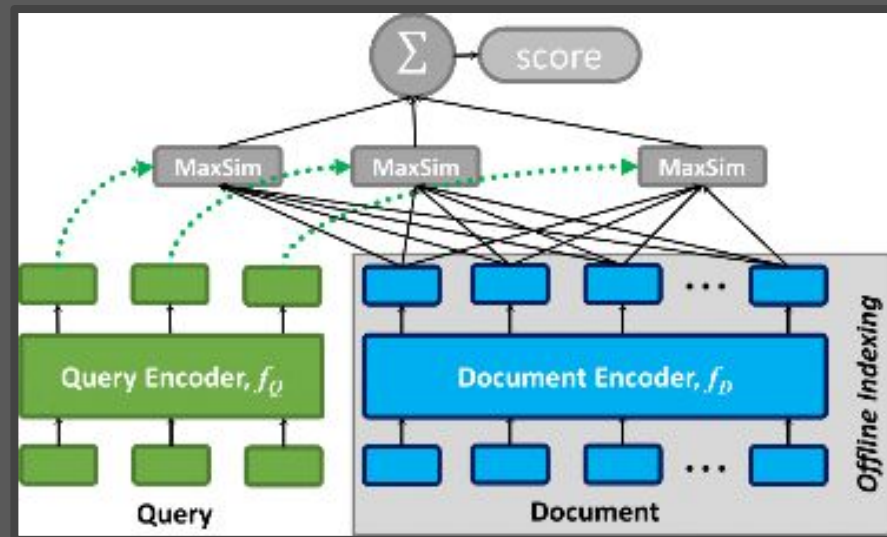
CoBERT

Проблема:

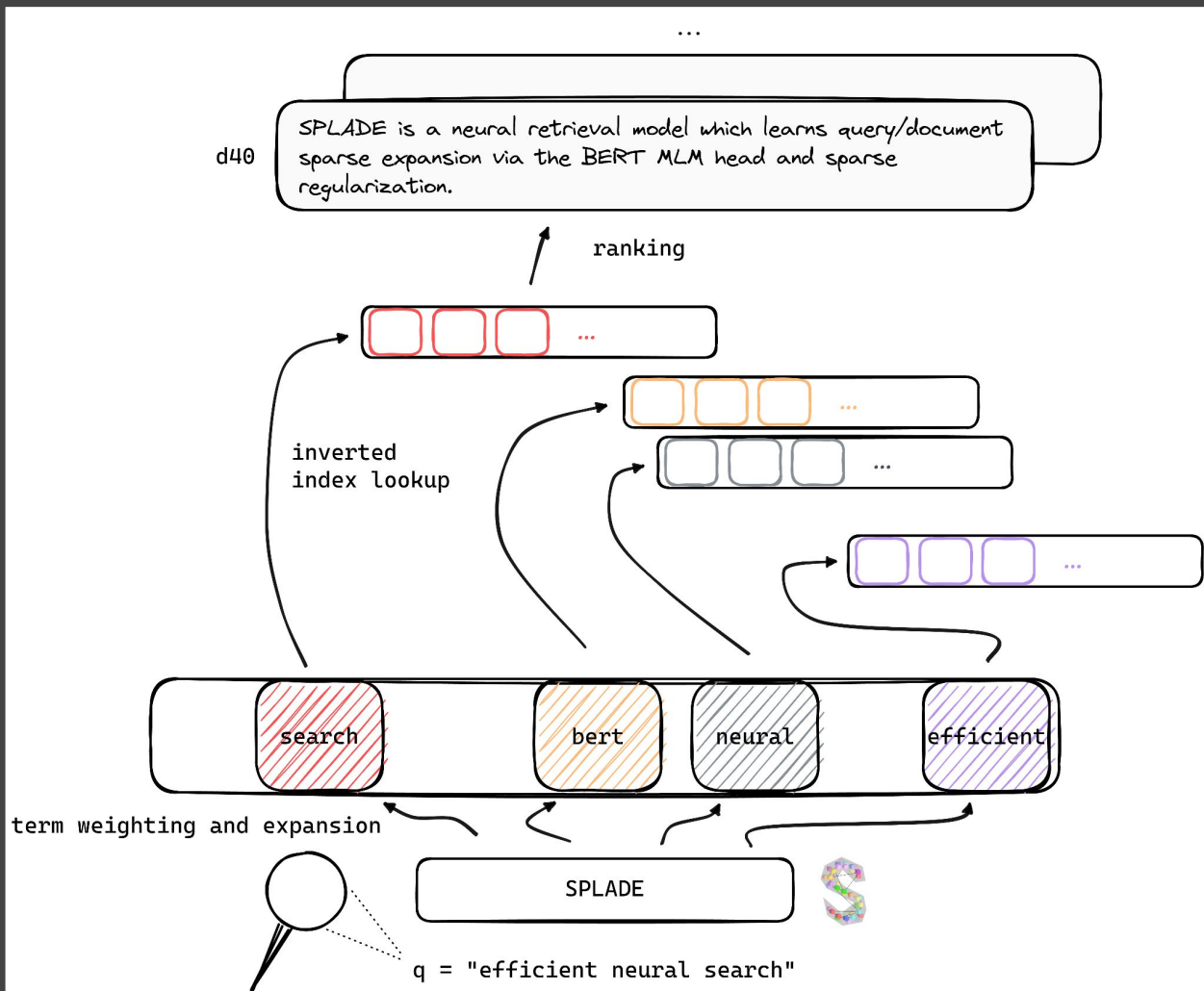
- Bi-encoder: теряет детали (single vector per document)
- Cross-encoder: слишком медленный (full encoding)
- Вопрос: Можно ли получить лучшее из обоих миров?

Решение:

- Сохраняем эмбединги КАЖДОГО слова отдельно
- Сравниваем слова запроса и документа "на лету"

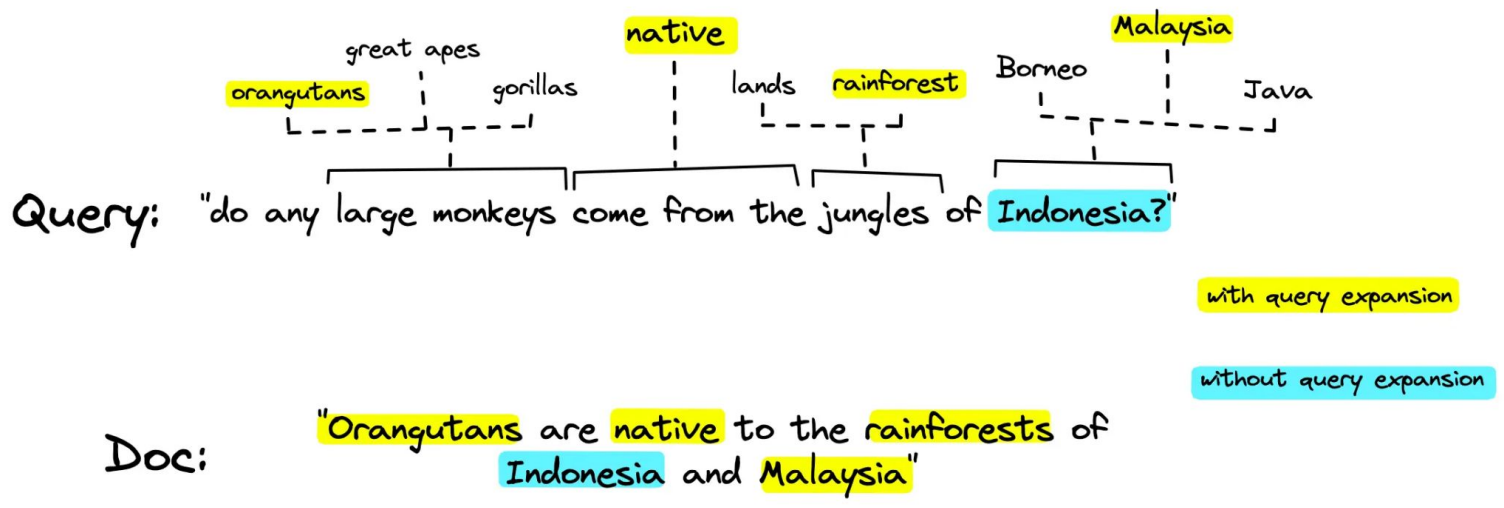


SPLADE



SPLADE

Neural model
создает sparse
vectors (как
BM25), но с
автоматическим
term expansion
(как semantic
search)



Hybrid Search

Query: "COVID-19 vaccine side effects"

BM25 находит:

1. "COVID-19 vaccine adverse reactions report" ✓
2. "Coronavirus vaccination complications" (пропустит "COVID-19")
3. "SARS-CoV-2 immunization" (пропустит "COVID-19")

Dense находит:

1. "Coronavirus vaccination complications" ✓ (semantic match)
2. "SARS-CoV-2 immunization side effects" ✓ (semantic)
3. "mRNA vaccine reactions" ✓ (conceptual)

Hybrid (объединенные):

1. "COVID-19 vaccine adverse reactions report" ✓ (exact match!)
2. "Coronavirus vaccination complications" ✓ (semantic)
3. "SARS-CoV-2 immunization side effects" ✓ (semantic + partial lexical)

Hybrid Search

Reciprocal Rank Fusion (RRF)

$$RRFscore(d \in D) = \sum_{r \in R} \frac{1}{k + r(d)}$$

D - set of docs

R - set of rankings as permutation on 1..|D|

K - typically set to 60 by default

Hybrid Search

Ранжирование BM25:

1. Документ А (скор: 15.2)
2. Документ В (скор: 12.1)
3. Документ С (скор: 10.5)
4. Документ D (скор: 8.3)
5. Документ Е (скор: 7.1)

Ранжирование Dense:

1. Документ С (скор: 0.92)
2. Документ А (скор: 0.89)
3. Документ Е (скор: 0.85)
4. Документ F (скор: 0.81)
5. Документ В (скор: 0.78)

Hybrid Search

Шаг 1: Вычисляем RRF для каждого документа

Документ А:

- Ранг в BM25: 1 $\rightarrow 1/(60+1) = 1/61 = 0.0164$
- Ранг в Dense: 2 $\rightarrow 1/(60+2) = 1/62 = 0.0161$
- $RRF(A) = 0.0164 + 0.0161 = 0.0325$

Документ В:

- Ранг в BM25: 2 $\rightarrow 1/62 = 0.0161$
- Ранг в Dense: 5 $\rightarrow 1/65 = 0.0154$
- $RRF(B) = 0.0161 + 0.0154 = 0.0315$

Документ С:

- Ранг в BM25: 3 $\rightarrow 1/63 = 0.0159$
- Ранг в Dense: 1 $\rightarrow 1/61 = 0.0164$
- $RRF(C) = 0.0159 + 0.0164 = 0.0323$

Документ D:

- Ранг в BM25: 4 $\rightarrow 1/64 = 0.0156$
- Ранг в Dense: нет $\rightarrow 0$
- $RRF(D) = 0.0156$

Документ Е:

- Ранг в BM25: 5 $\rightarrow 1/65 = 0.0154$
- Ранг в Dense: 3 $\rightarrow 1/63 = 0.0159$
- $RRF(E) = 0.0154 + 0.0159 = 0.0313$

Документ F:

- Ранг в BM25: нет $\rightarrow 0$
- Ранг в Dense: 4 $\rightarrow 1/64 = 0.0156$
- $RRF(F) = 0.0156$

Hybrid Search

Финальное ранжирование:

1. Документ A: $RRF = 0.0325$ ← Топ в обоих методах!
2. Документ C: $RRF = 0.0323$ ← №1 в dense, №3 в BM25
3. Документ B: $RRF = 0.0315$ ← №2 в BM25, №5 в dense
4. Документ E: $RRF = 0.0313$ ← В обоих топ-5
5. Документ D: $RRF = 0.0156$ ← Только в BM25
6. Документ F: $RRF = 0.0156$ ← Только в dense

Почему RRF работает:

1. Независимость от шкалы:

- BM25 скоры: 0-20
- Dense скоры: 0-1
- RRF не зависит от масштаба! Использует только ранги.

2. Устойчивость к выбросам:

- Если BM25 дал очень высокий скор из-за накрутки ключевых слов
- RRF смотрит только на позицию, не на абсолютный скор

3. Простота:

- Нет параметров для настройки ($k=60$ работает хорошо)
- Легко объяснить

Query Expansion

Проблема:

Запросы пользователей часто неоптимальны:

- Слишком короткие: "НС" (что именно?)
- Опечатки: "нейроная сть"
- Неполные: "обучить модель" (какую? как?)
- Неоднозначные: "python" (язык? змея? фильм?)

Решение: Трансформируем запрос перед поиском!

Query Expansion

Подход 1: Расширение запроса

Основная идея: Добавляем релевантные термины к исходному запросу

- Расширение синонимами (классика)
- Расширение через LLM

Query Expansion: HyDE

Запрос: "Что такое BERT?" -> LLM генерирует гипотетический документ -> LLM генерирует:

"BERT (Bidirectional Encoder Representations from Transformers)... -> Кодируем гипотетический документ -> Семантический поиск с этим вектором -> Возвращаем реальные документы

HyDE документ ближе к реальным документам потому что:

- Тот же домен
- Та же длина (примерно 100-200 токенов)
- Похожий стиль написания

Diversity: Избегаем дубликатов в результатах

Query: "python"

Top-5 результатов (ranked by relevance):

1. "Python programming tutorial" (score: 0.95)
2. "Python programming guide" (score: 0.94)
3. "Learn Python programming" (score: 0.93)
4. "Python programming for beginners" (score: 0.92)
5. "Complete Python programming course" (score: 0.91)

Проблема: Все про одно и то же! (Python программирование)

А что если пользователь искал:

- Python язык программирования? ✓
- Python змею? ✗
- Monty Python (фильм)? ✗
- Python фреймворк (web)? ✗

Diversity: Избегаем дубликатов в результатах

Решение: Maximal Marginal Relevance (MMR)

Основная идея:

- Выбираем документы итеративно
- Каждый новый документ должен быть:
 - Релевантен *query*
 - НЕ похож на уже выбранные

Баланс: Relevance vs Diversity

$$MMR = \lambda \cdot Relevance - (1 - \lambda) \cdot max_similarity_to_selected$$

LLM Reranking

LLM как judge релевантности

[query + doc] → BERT → score

[query + doc] → GPT-4 → "Is this relevant? Rate 1-10"

LLM Reranking

Подход: Listwise Ranking

Prompt example:

Given the query and list of documents, rank them by relevance.

Query: "machine learning tutorial"

Documents:

1. "PyTorch deep learning guide for beginners"
2. "NumPy array operations handbook"
3. "TensorFlow 2.0 complete tutorial"
4. "Introduction to neural networks"
5. "Pandas data manipulation guide"

Task: Reorder documents from most to least relevant.

Output format: [3, 1, 4, 2, 5]

Полезности

Stanford CS276 - Information Retrieval

Manning, Raghavan, Schütze - "Introduction to Information Retrieval"

E5: Wang et al. (2023)

BGE-M3: Chen et al. (2024)

BEIR: Thakur et al. (2021)

sentence-transformers

MS MARCO

MTEB